FitBuddy-AI Fitness Plan Generator using Gemini Models

Phase 4 – Project Planning

1. Introduction

Project planning organizes development activities into a practical sequence. For FitBuddy, planning is important because the application contains several connected components: web routes, database operations, HTML templates, AI modules, configuration, and testing. Development is therefore divided into smaller tasks so that each component can be implemented and checked before integration.

2. Development Sequence

The first task is to create the Python project and virtual environment. Required dependencies are then installed. After the environment is ready, the FastAPI application is created and the database layer is developed. Schemas are added to validate user input.

The next stage is AI integration. The workout generator is connected first, followed by the nutrition generator. After these basic generation functions work, feedback-based plan updating is implemented. The HTML templates and static resources are then connected to the backend. Administrative functions and the health endpoint are added before full testing.

The final activities are documentation preparation and project demonstration. This sequence reduces the chance of trying to debug every component at the same time.

3. Project Structure Plan

The planned structure contains app/main.py, app/routes.py, app/database.py, app/schemas.py, app/config.py, and the AI modules under app/ai/. The templates directory contains index.html, result.html, and all_users.html. A static directory is used for static resources.

The project also requires an appropriate Python virtual environment. During execution, Uvicorn runs app.main:app and watches the project for changes when reload mode is enabled.

4. Team and Task Planning

The project can be divided into responsibilities such as frontend/UI development, backend/API development, database work, Gemini AI integration, testing, documentation, and presentation. Although responsibilities may be divided among team members, every member should understand the complete workflow because the final demonstration depends on the integration of all modules.

5. Risk Identification

6.

Several risks were identified during planning. Package dependencies may be missing from the virtual environment. Incorrect module paths can produce import errors. Template or static directories may be missing or incorrectly configured. Gemini API requests can also be affected by quota or rate limits. Form handling requires python-multipart, and database configuration must be available before database operations can work.

Another risk is integration mismatch: a file may exist under app/ai/ while a route imports it as app.<module>. The planning process therefore includes repeated import and startup checks after major changes.

7. Testing and Milestone Plan

Testing is planned at multiple levels. Individual modules are checked first, followed by application startup, endpoint testing, database testing, AI generation testing, feedback testing, and final workflow testing. Swagger documentation at /docs is useful for checking the available endpoints and their input requirements.

A milestone is considered complete when the related feature can be executed without an import, dependency, routing, or template error.

8. Conclusion

The planning phase provides a roadmap for developing FitBuddy in an organized manner. It identifies development tasks, project structure, responsibilities, risks, and testing milestones. This plan supports smoother development and prepares the team for the final testing, documentation, and demonstration phases.

Phase Deliverables / Evidence

Item	Purpose	Status / Evidence
Development sequence	Orders implementation tasks	Documented
Risk list	Identifies likely issues	Documented
Testing milestones	Plans verification	Documented